ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Спецификация и архитектурное проектирование многоагентной системы»

Тема: Adaptive Multi-Agent LLM System - адаптивная многоагентная система для решения комплексных технических и исследовательских задач.

Выполнил: **Исмагилов Артём Азаматович**

Группа: **Б24-517**

2026

Содержание

1. Выбор и описание прикладной задачи **3**

1.1. Наименование и назначение системы 3
1.2. Постановка прикладной задачи 3
1.3. Цель системы 3
1.4. Пример прикладного сценария 3
1.5. Обоснование применения MAC 3
1.6. Требования к системе 3

2. Формальное описание многоагентной системы **4**

2.1. Состав системы 4
2.2. Orchestrator Agent - координатор 4
2.3. Adaptive Router Agent - адаптивный маршрутизатор 4
2.4. Research Agent - агент исследования 4
2.5. Coding Agent - агент программной разработки 4
2.6. Math Agent - агент математических расчётов 5
2.7. Critic Agent - агент проверки 5
2.8. Сводная характеристика агентов 5
2.9. Инфраструктурные компоненты 5
2.10. Формальная модель MAC 5
2.11. Матрица полномочий 6
2.12. Жизненный цикл задачи 6
2.13. Каналы связи и правила взаимодействия 6

3. Диаграмма ролей и граф коммуникаций **7**

3.1. Диаграмма ролей 7
3.2. Граф коммуникаций 7
3.3. Сценарий взаимодействия 8

4. Обоснование выбора архитектуры **8**

4.1. Тип архитектуры 8
4.2. Централизованная координация 8
4.3. Специализация агентов 8
4.4. Адаптивный выбор моделей 8
4.5. Message Bus 8
4.6. Пригодность архитектуры для последующих лабораторных работ 9
4.7. Ограничения выбранной архитектуры 9

Заключение **9**

1. Выбор и описание прикладной задачи

1.1. Наименование и назначение системы

Adaptive Multi-Agent LLM System - адаптивная многоагентная система, предназначенная для решения комплексных технических и исследовательских задач с использованием специализированных программных агентов и больших языковых моделей.

1.2. Постановка прикладной задачи

Решение комплексной технической задачи обычно включает поиск и анализ информации, разработку программного решения, математические расчёты и оптимизацию, проверку промежуточных результатов и подготовку итогового ответа. Одна универсальная языковая модель не всегда одинаково хорошо справляется со всеми перечисленными видами деятельности: они предъявляют различные требования к компетенциям, инструментам и вычислительным ресурсам.

Требуется спроектировать многоагентную систему, в которой специализированные агенты совместно выполняют пользовательский запрос. Система должна декомпозировать исходную задачу, распределять подзадачи между исполнителями, обеспечивать обмен сообщениями, контролировать выполнение и проверять результаты.

1.3. Цель системы

Разработка архитектуры адаптивной MAC, обеспечивающей совместное решение сложных технических и исследовательских задач посредством декомпозиции, специализированного выполнения подзадач, межагентного взаимодействия и проверки результатов.

1.4. Пример прикладного сценария

Пользователь запрашивает разработку алгоритма оптимизации маршрутов доставки: необходимо изучить существующие методы, реализовать алгоритм на Python, сравнить его с базовым решением и объяснить результаты. Research Agent изучает методы; Math Agent формализует задачу; Coding Agent реализует и тестирует алгоритм; Critic Agent проверяет результаты; Orchestrator объединяет материалы в итоговый ответ.

1.5. Обоснование применения MAC

Многоагентный подход позволяет распределить работу между специализированными участниками, выполнять независимые подзадачи параллельно, делегировать работу агентам с подходящими компетенциями и независимо проверять результаты. Адаптивный выбор моделей позволяет учитывать тип задачи, качество модели, задержку и стоимость её использования. Явные правила координации создают основу для исследования механизмов распределения задач и разрешения конфликтов.

1.6. Требования к системе

Код	Требование	Описание
R1	Приём запросов	Приём пользовательского запроса на естественном языке.
R2	Декомпозиция	Разбиение сложной задачи на подзадачи.
R3	Распределение	Назначение подзадач агентам по специализации.
R4	Взаимодействие	Обмен сообщениями и передача подзадач между агентами.
R5	Проверка	Проверка промежуточных и итоговых результатов.
R6	Адаптивность	Выбор языковых моделей для разных ролей.
R7	Контроль ресурсов	Контроль времени, числа запросов и токенов.
R8	Завершение	Формирование единого итогового ответа.
R9	Трассировка	Сохранение сообщений, статусов и результатов.
R10	Расширяемость	Добавление агентов и механизмов координации.

2. Формальное описание многоагентной системы

2.1. Состав системы

Система включает шесть логических агентов: Orchestrator Agent, Adaptive Router Agent, Research Agent, Coding Agent, Math Agent и Critic Agent. Инфраструктурные компоненты: Message Bus, LLM Client, Model Registry, Tool Layer, Task Store и Logger.

Агент представляет собой программную сущность, включающую языковую модель, инструкции, доступные инструменты, состояние и логику управления. Агент не тождественен используемой языковой модели: разные агенты могут использовать разные модели, а одна модель может применяться несколькими агентами.

2.2. Orchestrator Agent - координатор

Роль и цель. Управление выполнением пользовательского запроса, декомпозиция задачи, координация исполнителей и формирование итогового ответа.

Знания. Исходный запрос; структура задачи и подзадач; специализации агентов; статусы и зависимости; ограничения ресурсов; результаты исполнителей.

Состояния. IDLE - ожидание; PLANNING - планирование; DISPATCHING - распределение; MONITORING - контроль; INTEGRATING - объединение; COMPLETED - завершение; FAILED - ошибка.

Действия. Приём запросов; декомпозиция; создание подзадач; обращение к Router; назначение и переназначение исполнителей; контроль сроков; повторное выполнение; запрос проверки; формирование ответа.

Полномочия. Изменение структуры задач, назначение и отмена подзадач, установка приоритетов, контроль бюджета и завершение задачи. Не подменяет специализированных исполнителей.

2.3. Adaptive Router Agent - адаптивный маршрутизатор

Роль и цель. Выбор исполнителя и языковой модели для подзадачи с учётом компетенций и ресурсов.

Знания. Типы задач; специализации и доступность агентов; характеристики моделей; история выполнения; качество, задержка, стоимость и ограничения API.

Состояния. IDLE; ANALYZING; SELECTING; ASSIGNED; NO_CAPACITY.

Действия. Классификация подзадач; определение компетенций; оценка исполнителей; выбор модели; учёт ограничений; передача рекомендации Orchestrator.

Полномочия. Выбор исполнителя из зарегистрированного списка и модели из Model Registry; отказ при отсутствии подходящего исполнителя; предложение альтернативы. Не изменяет исходную задачу и не превышает общий бюджет.

2.4. Research Agent - агент исследования

Роль и цель. Поиск, сбор, анализ и структурирование информации.

Знания. Исследовательская подзадача; методы поиска; внешние источники; критерии достоверности; предыдущие результаты.

Состояния. IDLE; SEARCHING; ANALYZING; REQUESTING_HELP; REPORTING; COMPLETED.

Действия. Поиск информации; анализ документов; сравнение методов; формирование выводов; передача материалов; запрос помощи Coding Agent или Math Agent.

Полномочия. Выполнять исследования, обращаться к источникам, запрашивать помощь и возвращать результаты с источниками. Не изменяет глобальный план и не назначает задачи другим агентам самостоятельно.

2.5. Coding Agent - агент программной разработки

Роль и цель. Разработка и проверка программных компонентов.

Знания. Техническое задание; требования к реализации; языки и библиотеки; результаты исследований; данные и тесты.

Состояния. IDLE; DESIGNING; CODING; TESTING; DEBUGGING; COMPLETED.

Действия. Проектирование решения; написание кода; запуск тестов; анализ ошибок; исправление дефектов; передача результатов; запрос информации у Research и рекомендаций у Math.

Полномочия. Изменять код в выделенной области, запускать разрешённые инструменты, тестировать решение и запрашивать помощь. Не меняет системные ограничения и общий бюджет.

2.6. Math Agent - агент математических расчётов

Роль и цель. Математическая формализация, расчёты, моделирование и оптимизация.

Знания. Постановка задачи; методы и формулы; ограничения; входные данные; результаты вычислений.

Состояния. IDLE; FORMALIZING; CALCULATING; OPTIMIZING; VERIFYING; COMPLETED.

Действия. Формализация; выбор методов; вычисления; оптимизация; проверка; предоставление обоснований; взаимодействие с Coding и Research.

Полномочия. Выполнять вычисления в разрешённой среде, выбирать методы, запрашивать данные и возвращать результаты с пояснениями.

2.7. Critic Agent - агент проверки

Роль и цель. Контроль качества промежуточных и итоговых результатов, выявление ошибок и противоречий.

Знания. Исходная задача; требования; результаты исполнителей; критерии проверки; история исправлений.

Состояния. IDLE; REVIEWING; ISSUES_FOUND; APPROVED; REJECTED.

Действия. Проверка соответствия задаче; анализ логики; проверка кода и тестов; выявление противоречий; формирование замечаний; запрос доработки; подтверждение результата.

Полномочия. Принимать или отклонять результаты проверки, формировать замечания и рекомендовать повторное выполнение. Окончательное решение о завершении принимает Orchestrator.

2.8. Сводная характеристика агентов

Агент	Функция	Входные данные	Выходные данные
Orchestrator	Управление задачами	Запрос, сообщения, статусы	План, подзадачи, ответ
Router	Выбор исполнителя	Подзадача, характеристики агентов	Назначение исполнителя и модели
Research	Поиск и анализ	Исследовательская задача	Отчёт, источники, выводы
Coding	Разработка	Техническое задание, данные	Код, тесты, описание
Math	Расчёты и оптимизация	Математическая постановка	Формулы, вычисления, решение
Critic	Проверка	Результаты и критерии	Замечания или подтверждение

2.9. Инфраструктурные компоненты

Компонент	Назначение
Message Bus	Передача структурированных сообщений между агентами.
LLM Client	Унифицированный интерфейс взаимодействия с языковыми моделями.
Model Registry	Хранение моделей, характеристик и доступности.
Tool Layer	Доступ к поиску, вычислительной среде и файловым операциям.
Task Store	Хранение задач, статусов, зависимостей и результатов.
Logger	Запись сообщений и событий для анализа выполнения.

Система представляется кортежем $MAS = \langle A, T, S, M, C, R, P \rangle$, где A - множество агентов; T - множество задач и подзадач; S - множество состояний; M - множество сообщений; C - множество инструментов и ресурсов; R - правила взаимодействия; P - полномочия агентов.

Множество агентов: $A = \{O, RT, RS, CD, MA, CR\}$, где O - Orchestrator; RT - Router; RS - Research; CD - Coding; MA - Math; CR - Critic. Задача описывается структурой $t_i = \langle id, description, type, priority, status, parent, assignee, deadline \rangle$. Поле parent указывает на родительскую задачу, assignee - на назначенного исполнителя.

Состояния задачи: NEW, READY, ASSIGNED, RUNNING, WAITING, REVIEW, COMPLETED, FAILED, CANCELLED. Сообщение описывается структурой $m = \langle id, sender, receiver, type, task_id, payload, timestamp \rangle$.

2.11. Матрица полномочий

Операция	Orch.	Router	Research	Coding	Math	Critic
Создание подзадач	Да	Нет	Нет	Нет	Нет	Нет
Выбор исполнителя	Контроль	Да	Нет	Нет	Нет	Нет
Исследования	Нет	Нет	Да	Нет	Нет	Нет
Изменение кода	Нет	Нет	Нет	Да	Нет	Нет
Математические расчёты	Нет	Нет	Нет	Нет	Да	Нет
Проверка результатов	Контроль	Нет	Само	Само	Само	Да
Запрос помощи	Да	Нет	Да	Да	Да	Да
Изменение общего плана	Да	Нет	Нет	Нет	Нет	Нет
Завершение задачи	Да	Нет	Нет	Нет	Нет	Нет

2.12. Жизненный цикл задачи

После получения запроса Orchestrator выполняет декомпозицию и переводит задачу в READY. Router выбирает исполнителя, после чего задача получает статус ASSIGNED и переходит в RUNNING. При ожидании ответа другого агента задача может перейти в WAITING. Подготовленный результат направляется на проверку (REVIEW). При успешной проверке задача получает статус COMPLETED; при замечаниях может быть повторно назначена исполнителю. Ошибка приводит к FAILED, отмена - к CANCELLED.

Для предотвращения бесконечной передачи задач устанавливаются ограничения максимальной глубины делегирования, числа передач, времени выполнения и общего бюджета токенов и запросов. Для связи подзадач используется идентификатор родительской задачи.

2.13. Каналы связи и правила взаимодействия

Основной механизм коммуникации- обмен структурированными сообщениями через Message Bus. Централизованное взаимодействие используется для назначения задач и контроля выполнения. Межагентное взаимодействие используется для запросов информации и делегирования специализированных подзадач. Например, Research Agent может запросить Coding Agent реализовать изучаемый алгоритм. Coding Agent возвращает код и результаты тестирования, после чего Research Agent использует их в исследовании. Orchestrator сохраняет контроль над общим состоянием, бюджетом и итоговой сборкой.

Тип сообщения	Назначение
TASK_ASSIGN	Назначение задачи агенту.
TASK_ACCEPT	Подтверждение принятия задачи.
TASK_REJECT	Отказ от выполнения.
HELP_REQUEST	Запрос помощи другого агента.
RESULT	Передача результата.
REVIEW_REQUEST	Запрос проверки.
REVIEW_FEEDBACK	Передача замечаний.
TASK_UPDATE	Изменение статуса задачи.
TASK_CANCEL	Отмена задачи.
ERROR	Сообщение об ошибке.

Каждая задача имеет уникальный идентификатор; сообщение содержит отправителя, получателя и идентификатор задачи; агент действует в пределах полномочий; межагентный запрос не отменяет контроль Orchestrator; общий бюджет контролируется Orchestrator; результаты для итогового ответа проходят проверку; ошибки передаются через Message Bus; повторные попытки и делегирование ограничиваются по количеству и времени.

3. Диаграмма ролей и граф коммуникаций

Диаграмма отражает функциональное распределение ролей. Orchestrator управляет выполнением запроса, Router выбирает исполнителей, Research, Coding и Math выполняют специализированные подзадачи, Critic проверяет результаты. Инфраструктурные компоненты обеспечивают обмен сообщениями, доступ к моделям и инструментам, а также хранение состояния.

3.1. Диаграмма ролей

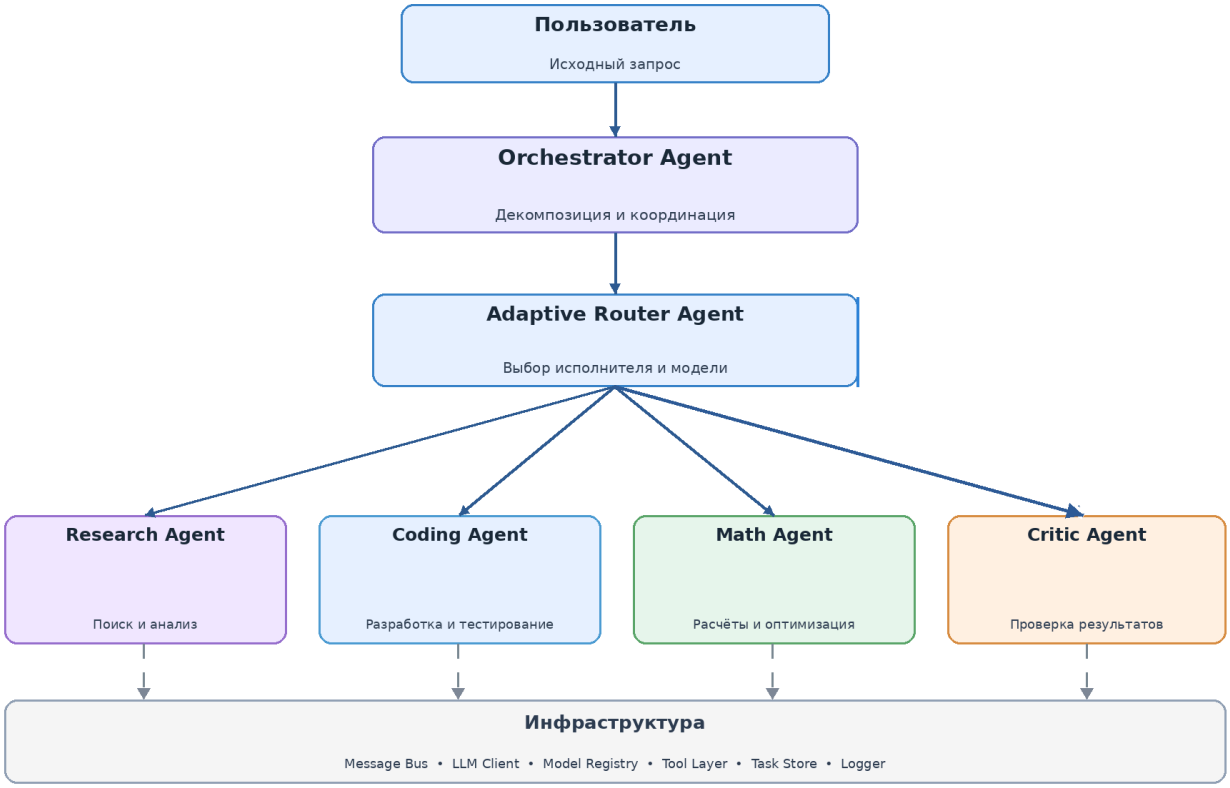


Рисунок 1 - Диаграмма ролей многоагентной системы

3.2. Граф коммуникаций

Orchestrator взаимодействует со всеми агентами. Router передаёт назначения специализированным исполнителям. Research, Coding и Math обмениваются запросами и результатами; исполнители направляют результаты Critic для проверки.

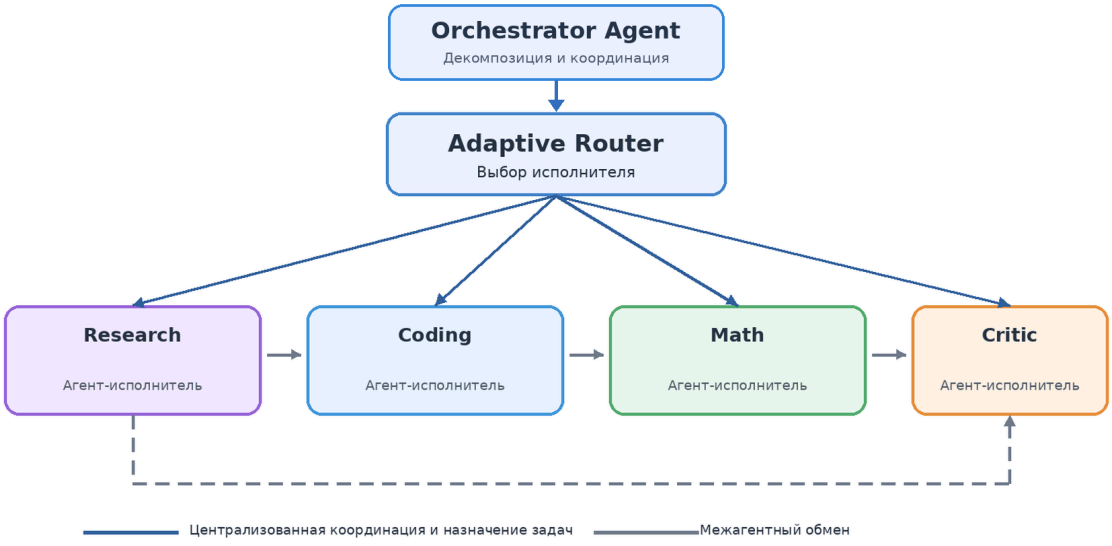


Рисунок 2 - Граф коммуникаций агентов

3.3. Сценарий взаимодействия

При решении задачи оптимизации маршрутов пользовательский запрос поступает Orchestrator. Координатор формирует подзадачи и обращается к Router. Research изучает методы, Math формализует задачу и ограничения. Coding реализует алгоритм; при необходимости он запрашивает у Math уточнение математической модели. Результаты направляются Critic. Если обнаружены ошибки, Orchestrator назначает доработку и инициирует повторную проверку. После подтверждения Orchestrator объединяет результаты и формирует ответ пользователю.

4. Обоснование выбора архитектуры

4.1. Тип архитектуры

Выбрана гибридная LLM-based многоагентная архитектура с централизованным координатором и поддержкой межагентного взаимодействия. Интеллектуальная обработка задач выполняется с использованием больших языковых моделей; распределение задач, управление состояниями, контроль полномочий и ограничений задаются явной программной логикой; коммуникация осуществляется посредством структурированных сообщений.

4.2. Централизованная координация

Централизованная координация упрощает контроль над сложной задачей. Orchestrator располагает информацией о составе подзадач, зависимостях и состояниях, поэтому может обнаруживать задержки, переназначать задачи и контролировать ограничения. Полная централизация коммуникаций снизила бы гибкость, поэтому допускается межагентный обмен при сохранении общего контроля Orchestrator.

4.3. Специализация агентов

Специализация позволяет ориентировать инструкции и инструменты на конкретные виды деятельности, независимо оценивать качество компонентов, заменять модели без изменения всей архитектуры и сравнивать способы распределения задач. Отдельная языковая модель не обязана соответствовать каждому агенту: одна модель может использоваться несколькими ролями, если удовлетворяет их требованиям.

4.4. Адаптивный выбор моделей

Для каждой роли предусматривается конфигурационный выбор провайдера и модели. Унифицированный LLM Client скрывает общие детали взаимодействия с API, а Model Registry хранит идентификаторы и характеристики моделей. Различия провайдеров в форматах запросов, поддержке инструментов и структурированных ответов учитываются адаптерами и проверкой возможностей модели.

4.5. Message Bus

Message Bus отделяет логику агентов от механизма доставки сообщений. Это позволяет добавлять новых участников, регистрировать взаимодействия, анализировать историю выполнения, реализовывать разные протоколы координации и моделировать задержки и ошибки доставки.

Пример конфигурации моделей:

```
agents:
  orchestrator: {provider: openrouter, model: MODEL_ID_1}
  router: {provider: openrouter, model: MODEL_ID_2}
  research: {provider: openrouter, model: MODEL_ID_3}
  coding: {provider: openrouter, model: MODEL_ID_4}
  math: {provider: openrouter, model: MODEL_ID_5}
  critic: {provider: openrouter, model: MODEL_ID_6}
```

4.6. Пригодность архитектуры для последующих лабораторных работ

Работа	Предмет исследования	Архитектурные предпосылки
ЛР-2	Сравнение централизованной диспетчеризации, контрактной сети, аукциона или распределённого делегирования.	Router, Task Store, Message Bus, назначения исполнителей и регистрация событий.
ЛР-3	Конфликты интересов, конкурирующие заявки, дефицит ресурсов, переговоры и арбитраж.	Приоритеты задач, полномочия, ограничения ресурсов и межагентные сообщения.
ЛР-4	Недостоверные сообщения, отказы инструментов, циклическое делегирование и противоречивые указания.	Контроль полномочий, трассировка сообщений, лимиты делегирования и обработка ошибок.

4.7. Ограничения архитектуры

К ограничениям относятся возможность возникновения узкого места в Orchestrator, увеличение задержки и числа сообщений при межагентном взаимодействии, зависимость качества от языковых моделей, недоступность внешних API, вероятность ошибочных или противоречивых ответов, а также необходимость явных ограничений для предотвращения циклов делегирования. Эти факторы подлежат дальнейшему исследованию.

Заключение

В лабораторной работе спроектирована многоагентная система Adaptive Multi-Agent LLM System для решения комплексных технических и исследовательских задач. Сформулирована прикладная задача и определены требования к системе. Выполнено формальное описание шести агентов: Orchestrator, Adaptive Router, Research, Coding, Math и Critic; определены их роли, цели, знания, состояния, действия и полномочия. Описаны инфраструктурные компоненты, структура сообщений, каналы связи и правила взаимодействия.

В качестве архитектуры выбрана гибридная LLM-based MAC с централизованным координатором, адаптивным выбором исполнителей и возможностью межагентного взаимодействия. Представлены диаграмма ролей, граф коммуникаций и сценарий выполнения задачи. Архитектура предусматривает возможность исследования механизмов распределения задач и координации, переговоров и разрешения конфликтов, а также надёжности системы в последующих лабораторных работах.

Полученная спецификация определяет состав и принципы взаимодействия компонентов и служит архитектурной основой для дальнейшей реализации многоагентной системы.